

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_6e519d4b-9e6\agent-ac2034eb2f21f1191.jsonl`
- SHA-256: `e4d53bd1b049cf5b99404ddd98c4159cc62ce51a17a2fa5d288ded06d990e3df`
- Source modified: `2026-06-22T14:46:27+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_6e519d4b-9e6`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T14:44:22.146Z)

Read C:/Users/avidu/Projects/bhi-cloudrun/backend/api/job_worker.py fully (_run_claimed_job + helpers) and how the segmenter persists results. Map EXACTLY how the API job flow runs a job IN-PROCESS today: claim_due_job ? build segmenter ? segmenter.process_video ? how rally segments land in the SQLite DB (Database.add_video / play_segments), the result dict shape stored on the job, the status transitions (queued/running/done/failed), and how GET /api/query reads those segments afterward. Quote file:line. This is where the cloud-dispatch branch must slot in.

2. assistant (2026-06-22T14:44:25.036Z)

I'll start by pulling the bhi-cloudrun repo to ensure freshness, then read the target file and trace the flow.

3. user (2026-06-22T14:44:28.915Z)

You are not currently on a branch.
Please specify which branch you want to merge with.
See git-pull(1) for details.

```
git pull <remote> <branch>
```

```
---STATUS---
```

HEAD (no branch)

```
---BRANCH---
```

```
df3b225 docs(nightly): one-stop operations manual for the nightly engine-regression (#325)
```

4. user (2026-06-22T14:44:28.938Z)

```
1      """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
2      P3).
3
4      Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
5      The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
6      that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
7      re-claimed when due ? with no central scheduler (the DB is the clock).
8
9      The monkeypatch contract these functions rely on (and that tests depend on):
10     every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
11     `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
12     `backend.main` module object at call-time (`import backend.main as _main`), never as a
13     bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
14     ...)`` still intercepts the inter-function calls (e.g. a parked job arms the `patched`
15     `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
16     lazily
```

```

15     inside the functions also avoids the circular import at module load (main.py imports
this
16     module; this module must not import main at import time).
17     """
18     import logging
19     import threading
20     import traceback
21     from concurrent.futures import ThreadPoolExecutor
22     from typing import Any, Dict, Optional
23
24     logger = logging.getLogger(__name__)
25
26     # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
27     # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
28     # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
29     # already parallelize their AI handoff internally via their own pools.
30     _job_executor: Optional[ThreadPoolExecutor] = None
31
32
33     def _get_executor() -> ThreadPoolExecutor:
34         """Lazy module-global executor; tests monkeypatch this for inline execution."""
35         global _job_executor
36         if _job_executor is None:
37             _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
38         return _job_executor
39
40
41     class JobWaiting(Exception):
42         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
43         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`,
44         NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
45         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
46         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
47
48         The wiring is additive: the production segmenter path never raises this today (zero
49         behaviour change), so a job runs exactly as before. The hook is what a later slice
50         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
51         """
52
53         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
54             super().__init__(f"job parked for {delay_sec:.0f}s")
55             self.delay_sec = max(0.0, float(delay_sec))
56             self.progress = progress
57
58
59     def _job_to_request(job: Dict[str, Any]):
60         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
61         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
62         the original submit. The row stores exactly the ProcessRequest fields."""
63         import backend.main as _main
64         return _main.ProcessRequest(
65             video_path=job["video_path"],
66             player_pool=job.get("player_pool"),
67             max_frames=job.get("max_frames"),
68             segmenter=job.get("segmenter"),
69         )
70
71
72     def _run_claimed_job(job: Dict[str, Any]) -> None:
73         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its

```

```

74     terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
75
76     Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
77     the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
78     result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
79     job self-scheduled and will be re-claimed when `next_poll_at` is due.
80     """
81     import backend.main as _main
82     db_instance = _main.db_instance
83     job_id = job["id"]
84     req = _main._job_to_request(job)
85     try:
86         result = _main._execute_processing(
87             req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
88         if result.get("video_id"):
89             db_instance.set_job_video_id(job_id, result["video_id"])
90             db_instance.mark_job_done(job_id, result)
91             _maybe_record_job_cost(job, result) # M8 cost ledger (default-OFF)
92     except _main.JobWaiting as w:
93         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
94         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
95         db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
96     except Exception as e:
97         logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
98         db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
99
100
101     def _maybe_record_job_cost(job: Dict[str, Any], result: Dict[str, Any]) -> None:
102         """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
103         ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
104         (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
105         persist it.
106
107         ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
108         ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
109         FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
110         + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
111         With the placeholder pricebook the cost is 0 regardless.
112
113         Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
114         claimed row (carries ``owner_id``); ``result`` is the pipeline output."""
115         import backend.main as _main
116
117         billing_cfg = getattr(_main.global_config, "billing", None)
118         if not billing_cfg or not getattr(billing_cfg, "enabled", False):
119             return
120         try:
121             usage = (result or {}).get("usage") or {}
122             _main.db_instance.record_job_cost(
123                 job["id"], usage=usage, pricebook_version=billing_cfg.pricebook_version,
124                 gpu_type=usage.get("gpu_type"), compute_backend=(result or
{}).get("compute_backend"),
125                 owner_id=job.get("owner_id"), margin=billing_cfg.margin,
126             )
127             logger.info("Recorded cost for job %s (pricebook=%s).", job["id"],
billing_cfg.pricebook_version)
128         except Exception as e: # never wedge a completed job on bookkeeping

```

```

129         logger.warning("Cost recording for job %s failed (non-fatal): %s",
job.get("id"), e)
130
131
132     def _drain_due_jobs() -> int:
133         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
134         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
135
136         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
137         up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
138         a single-worker box keeps making progress with no central timer (the DB is the
clock).
139         Returns the number of jobs that reached a terminal/parked state this tick.
140         """
141         import backend.main as _main
142         ran = 0
143         while True:
144             job = _main.db_instance.claim_due_job()
145             if job is None:
146                 break
147             ran += 1
148             _main._run_claimed_job(job)
149             # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
150             # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
151             # on the next submit/drain. Best-effort ? never raises into the worker.
152             _main._schedule_next_drain_if_waiting()
153             return ran
154
155
156     def _schedule_next_drain_if_waiting() -> None:
157         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
158         self-scheduled job resumes with no further client submit. The drain itself re-checks
159         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
160         import backend.main as _main
161         try:
162             delay = _main.db_instance.seconds_until_next_due()
163         except Exception as e: # pragma: no cover - defensive; never wedge the worker
164             logger.error(f"Could not compute next-due delay: {e}")
165             return
166         if delay is None:
167             return # nothing waiting
168         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
169         _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)))
170
171
172     def _arm_wake_timer(delay_sec: float) -> None:
173         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
174         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
175         import backend.main as _main
176         timer = threading.Timer(delay_sec, lambda:
_main._get_executor().submit(_main._drain_due_jobs))
177         timer.daemon = True
178         timer.start()
179
180

```

```
181
182     #: A parked job further out than this is re-checked by a capped wake rather than one
long
183     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
184     _MAX_DRAIN_WAKE_SEC = 60.0
185
```

5. assistant (2026-06-22T14:44:32.395Z)

The file resolves everything through `backend.main`. Let me read the actual implementations there: `\_execute\_processing`, `claim\_due\_job`, `mark\_job\_done`, the segmenter build, and how segments persist + how `/api/query` reads them.

6. user (2026-06-22T14:44:33.060Z)

backend\main.py

7. user (2026-06-22T14:44:34.653Z)

annotations
api
assets
billing.py
compute
config
eval
features
infrastructure
main.py
pipeline
providers
reporting
results.py
sports
storage
tools
ui
utils
worker
---API---
__init__.py
auth.py
job_worker.py
static
uploads.py
---INFRA---
database.py
execution_policy.py

8. user (2026-06-22T14:44:39.142Z)

```
259: class ProcessRequest(BaseModel):
439: def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
442:     This is the former synchronous /api/process body, unchanged in behavior, now run on
476:         # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
478:         db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump() for r
in val_results])
489:         db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for r
in val_results])
501:         "play_segments": [],
513:         result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
530:         "play_segments": [],
543:         "play_segments": segments,
```

```

559:@app.post("/api/process", status_code=202)
560:def process_video(req: ProcessRequest, request: Request):
561:@app.post("/api/query")
562:def query_play_segments(req: QueryRequest):
563:    segments = db_instance.query_play_segments(req.video_id, filters)
564:    return {"play_segments": segments}
565:    all_segments = db_instance.query_play_segments(req.video_id, {})
566:    # video ingested via /api/process from a URI, or a plain local path. Resolve it to a
567:    LOCAL file
568:    # for a bucket URI (the same seam the process path uses, _execute_processing
569:    main.py:159).
570:    db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r in
571:    val_results])
572:    result = segmenter.process_video(video_id, local_video,
573:    max_frames=args.max_frames)

```

9. user (2026-06-22T14:44:42.044Z)

```

259    class ProcessRequest(BaseModel):
260        video_path: str
261        player_pool: Optional[List[str]] = None
262        max_frames: Optional[int] = None
263        segmenter: Optional[str] = None
264        # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
265        for PMF
266        # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
267        clients).
268        locale: Optional[str] = None
269
270    class QueryRequest(BaseModel):
271        video_id: str
272        server: Optional[str] = None
273        receiver: Optional[str] = None
274        duration_min: Optional[float] = None
275        event_type: Optional[str] = None
276
277    class CompileRequest(BaseModel):
278        video_id: str
279        segment_ids: List[int]
280        output_filename: str
281        # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel
282        watermark
283        # (text + script font). Optional ? absent/unknown keeps the configured default (en
284        behavior).
285        locale: Optional[str] = None
286
287    def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
288        """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
289
290        On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
291        fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
292        and
293        keep the prior good results intact ? a failed/empty re-run never destroys prior
294        data.
295        """
296        if segments:
297            db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
298        else:
299            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
300
301    # --- API Endpoints ---
302    @app.get("/api/config")
303    def get_config():

```

10. user (2026-06-22T14:44:42.422Z)

```
439     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
440         """The full hash ? validate ? segment ? report pipeline for one video.
441
442         This is the former synchronous /api/process body, unchanged in behavior, now run on
443         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
444         the sync endpoint used to return (UI compatibility); the per-video observability
445         report is still emitted in `finally:` and grafted as response["reports"].
446
447         `progress` is an optional callable(stage: str) used to surface coarse job progress.
448         Raises on unexpected internal errors ? the caller records those as a job failure
449         (after this function has already restored the pre-run segments, see below).
450         """
451         notify = progress or (lambda stage: None)
452
453         notify("hashing")
454         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
455         # (identity ? # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
456         # original # req.video_path (the source URI) is kept for the DB record + report; only the file-
457         # reading # consumers (hash / validators / segmenter) use the resolved local path.
458         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
459 "storage", None))
460         video_id = compute_video_id(local_video)
461         was_reingest = db_instance.get_video(video_id) is not None
462
463         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
464         # report)
465         notify("validating")
466         logger.info(f"Running validations for {req.video_path}...")
467         val_results, val_context = registry.run_all_with_context(local_video, global_config)
468         failures = [r for r in val_results if not r.passed]
469
470         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
471         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
472 "motion")
473         seg_name = req.segmenter or default_seg
474         segmenter_actual = None
475         segmentation_ran = False
476         response: Optional[Dict[str, Any]] = None
477         try:
478             if failures:
479                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
480                 # status; it no longer destroys the video's prior good segments.
481                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
482 for r in val_results])
483                 response = {
484                     "video_id": video_id,
485                     "status": "failed",
486                     "message": "Video failed validation checks.",
487                     "results": [r.model_dump() for r in val_results],
488                 }
489             return response
490
491         # 2. Run segmenter & indexer
492         logger.info("[API] Video passed checks. Segmenting and indexing...")
493         db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
494 r in val_results])
495
496         segmenter_cls = segmenter_registry.get(seg_name)
497         if not segmenter_cls:
498             # Submit-time validation already 400s unknown names; this is the defensive
```

```

494         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
495         available = list(segmenter_registry.list_available().keys())
496         response = {
497             "video_id": video_id,
498             "status": "failed",
499             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
500             "results": [r.model_dump() for r in val_results],
501             "play_segments": [],
502         }
503         return response
504
505         logger.info(f"[API] Using segmenter: {seg_name}")
506         notify(f"segmenting ({seg_name})")
507         segmenter_actual = seg_name
508         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
509         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
510         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
511         segmenter = segmenter_cls(db_instance, global_config)
512         try:
513             result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
514         except Exception:
515             # A raising segmenter must not leave half-written rows: restore the pre-run
516             # state (same destroy-after-confirm contract as the Failure branch), then
let
517             # the job worker record the failure.
518             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
519             raise
520         segmentation_ran = True
521
522         if isinstance(result, Failure):
523             logger.error(f"[API] Segmenter failed: {result.message}")
524             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
525             response = {
526                 "video_id": video_id,
527                 "status": "failed",
528                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
529                 "results": [r.model_dump() for r in val_results],
530                 "play_segments": [],
531             }
532             return response
533
534         segments = result.value if hasattr(result, "value") else result
535         notify("finalizing")
536         _finalize_reingest(video_id, segments, play_wm, cand_wm)
537
538         response = {
539             "video_id": video_id,
540             "status": "success",
541             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
542             "results": [r.model_dump() for r in val_results],
543             "play_segments": segments,
544         }
545         return response
546     finally:
547         # Always emit the per-video observability report (next to the video, or to
548         # report.output_dir). Never raises. Surface the paths in the response.
549         paths = emit_indexer_report(
550             db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,

```



```

551         val_results=val_results, val_context=val_context,
552         segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
553         was_reingest=was_reingest, segmentation_ran=segmentation_ran,
554     )
555     if paths and isinstance(response, dict):
556         response["reports"] = paths
557
558
559 @app.post("/api/process", status_code=202)
560 def process_video(req: ProcessRequest, request: Request):
561     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
562
563     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
564     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
565     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
566     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
567     """
568     # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
569     # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
570     # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
571     try:
572         owner_id = edge_auth.resolve_tenant(request.headers, global_config)
573     except edge_auth.EdgeAuthError as e:
574         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
575
576     allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
577     try:
578         validate_input_path(req.video_path, allowed_root=allowed_root)
579         # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
580         # scheme, which we map to a clean 400 (an unknown scheme must never 500).
581         input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
582     except ValueError as e:
583         raise HTTPException(status_code=400, detail=str(e)) from e
584     # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
585     # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
586     # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
587     # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
588     # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
589     # rejects a non-local URI as out-of-root.
590     if input_ref.backend == "local" and not os.path.exists(input_ref.key):
591         raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
592     if req.segmenter and not segmenter_registry.get(req.segmenter):
593         available = list(segmenter_registry.list_available().keys())
594         raise HTTPException(
595             status_code=400,
596             detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}")
597     )
598

```

11. assistant (2026-06-22T14:44:45.019Z)

Now let me read the rest of `/api/process` (job creation), the `/api/query` endpoint, and the

segmenter's `process\_video` to see how segments persist.

12. user (2026-06-22T14:44:45.999Z)

```
598
599     job_id = uuid.uuid4().hex
600     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
601                                   req.player_pool, req.max_frames, owner_id=owner_id,
602                                   locale=req.locale):
603         raise HTTPException(status_code=500, detail="Failed to persist job record.")
604     _get_executor().submit(_drain_due_jobs)
605     return JSONResponse(
606         status_code=202,
607         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
608     )
609
610
611     @app.get("/api/jobs/{job_id}/cost")
612     def get_job_cost(job_id: str):
613         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
614         truth (matches
615         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
616         ``cost_usd`` is
617         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
618         cost = db_instance.get_job_cost(job_id)
619         if cost is None:
620             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
621         fx = global_config.billing.fx_inr_per_usd
622         usd = cost.get("cost_usd")
623         cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
624         cost["fx_inr_per_usd"] = fx
625         return cost
626
627     @app.get("/api/videos/{video_id}/cost")
628     def get_video_cost(video_id: str):
629         """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
630         ``total_cost_inr`` + the
631         per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
632         several."""
633         agg = db_instance.get_video_cost(video_id)
634         fx = global_config.billing.fx_inr_per_usd
635         agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
636         agg["fx_inr_per_usd"] = fx
637         return agg
638
639     @app.get("/api/jobs/{job_id}")
640     def get_job(job_id: str):
641         """Job state: {status, progress, result, reports}. `status` = execution state
642         (queued|running|done|failed); the pipeline verdict is result["status"]."""
643         job = db_instance.get_job(job_id)
644         if not job:
645             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
646         result = job.get("result")
647         return {
648             "job_id": job["id"],
649             "status": job["status"],
650             "progress": job.get("progress"),
651             "video_id": job.get("video_id"),
652             "error": job.get("error"),
653             "result": result,
654             "reports": (result or {}).get("reports"),
655             "created_at": job.get("created_at"),
656             "started_at": job.get("started_at"),
```

```

655         "finished_at": job.get("finished_at"),
656     }
657
658     # --- Chunked upload (B2, PR-2) -----
659     # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
660     # in
661     # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
662     # via
663     # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
664     # sequential-part logic, and abort cleanup ? is unchanged.
665     from backend.api import uploads as uploads_api
666
667     app.include_router(uploads_api.router)
668
669     # --- Highlight download (B3, PR-2) -----
670
671     @app.get("/api/highlights/{video_id}/{filename}")
672     def download_highlight(video_id: str, filename: str):
673         """Stream a compiled highlight reel ? `filename` is the bare name given to
674         /api/compile
675         (which only writes into output_dir; the browser previously got back an unreachable
676         server-local path)."""
677         try:
678             name = safe_output_filename(filename)
679         except ValueError as e:
680             raise HTTPException(status_code=400, detail=str(e)) from e
681         if not db_instance.get_video(video_id):
682             raise HTTPException(status_code=404, detail="Indexed video record not found.")
683         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
684         or "output"
685         path = os.path.join(output_dir, name)
686         try:
687             path = resolve_under_root(path, output_dir)
688         except ValueError as e:
689             raise HTTPException(status_code=400, detail=str(e)) from e
690         if not os.path.isfile(path):
691             raise HTTPException(status_code=404,
692                                detail=f"No compiled file named '{name}'. Compile first via
693                                /api/compile.")
694         media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
695         return FileResponse(path, media_type=media, filename=name)
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714

```

```

715         if not matched_segments:
716             raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
717

```

13. assistant (2026-06-22T14:44:49.160Z)

Now let me look at the segmenter's `process\_video` and how it persists segments (`add\_video` / `play\_segments` / `add\_play\_segments`), plus the relevant `Database` methods.

14. user (2026-06-22T14:44:51.628Z)

```

24:     def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None,
max_frames: int = None) -> "Result[List[Dict[str, Any]]]":

```

15. user (2026-06-22T14:44:53.269Z)

```

95:         """Initializes tables for videos, play_segments, and events."""
172:         """Create the videos / play_segments / events base tables (idempotent)."""
186:         CREATE TABLE IF NOT EXISTS play_segments (
208:             FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
233:             FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON DELETE
SET NULL
405:     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
424:     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
432:         p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE video_id =
?",
438:     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
441:         """Delete play/candidate segments by id range (events cascade from play_segments).
450:         cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
452:         cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
459:     def add_play_segment(self, video_id: str, start_time: float, end_time: float, server:
Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]] = None)
-> Optional[int]:
464:         "INSERT INTO play_segments (video_id, start_time, end_time, duration,
server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
564:     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) -> List[Dict[str,
Any]]:
572:         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
623:     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
659:     def set_job_progress(self, job_id: str, progress: str) -> bool:
665:     def set_job_video_id(self, job_id: str, video_id: str) -> bool:
671:     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
681:     def mark_job_failed(self, job_id: str, error: str) -> bool:
689:     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
759:     def get_job_cost(self, job_id: str) -> Optional[Dict[str, Any]]:
802:     def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] = None)
-> bool:
831:     def claim_due_job(self) -> Optional[Dict[str, Any]]:

```

16. assistant (2026-06-22T14:44:56.750Z)

Let me read the segmenter base `process\_video`, the play_segments schema + jobs schema, and the key Database methods (add_play_segment, query_play_segments, create_job, claim_due_job, mark_job_done).

17. user (2026-06-22T14:44:58.177Z)

```

1     from abc import ABC, abstractmethod
2     from typing import List, Dict, Any, Type, Optional
3     from backend.infrastructure.database import Database

```

```

4         from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6         class BasePlaySegmenter(ABC):
7             def __init__(self, db: Database, config: Dict[str, Any]):
8                 self.db = db
9                 self.config = config
10
11             @property
12             @abstractmethod
13             def name(self) -> str:
14                 """Unique identifier for this segmenter."""
15                 pass
16
17             @property
18             @abstractmethod
19             def description(self) -> str:
20                 """Human-readable explanation of this segmenter."""
21                 pass
22
23             @abstractmethod
24             def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25                 """
26                 Processes a video to detect play segments, populates SQLite database,
27                 and returns a Result.
28
29                 Use isinstance(result, Failure) (or result.is_success()) to distinguish
30                 "no segments found" from "something went wrong" (missing key, healthcheck
31                 failure, etc.). This is the start of the standardized error/result contract.
32                 """
33                 pass
34
35         class SegmenterRegistry:
36             def __init__(self):
37                 self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39             def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40                 """Registers a new segmenter class under its name property."""
41                 try:
42                     temp_inst = segmenter_cls(None, {})
43                     name = temp_inst.name
44                 except Exception as e:
45                     raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46                 self._segmenters[name] = segmenter_cls
47
48             def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49                 return self._segmenters.get(name)
50
51             def list_available(self) -> Dict[str, str]:
52                 """Returns a dict of registered segmenter names and their descriptions."""
53                 res = {}
54                 for name, cls in self._segmenters.items():
55                     try:
56                         temp_inst = cls(None, {})
57                         desc = temp_inst.description
58                     except Exception:
59                         desc = "No description available."
60                 res[name] = desc

```

18. user (2026-06-22T14:44:58.742Z)

```

172         """Create the videos / play_segments / events base tables (idempotent)."""
173         # Videos Table

```

```

174         cursor.execute("""
175             CREATE TABLE IF NOT EXISTS videos (
176                 id TEXT PRIMARY KEY,
177                 filepath TEXT NOT NULL,
178                 processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
179                 status TEXT NOT NULL,
180                 validation_results TEXT
181             )
182         """)
183
184     # Play Segments Table (Extensible metadata JSON)
185     cursor.execute("""
186         CREATE TABLE IF NOT EXISTS play_segments (
187             id INTEGER PRIMARY KEY AUTOINCREMENT,
188             video_id TEXT NOT NULL,
189             start_time REAL NOT NULL,
190             end_time REAL NOT NULL,
191             duration REAL NOT NULL,
192             server TEXT,
193             receiver TEXT,
194             metadata TEXT,
195             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
196         )
197     """)
198
199     # Events Table
200     cursor.execute("""
201         CREATE TABLE IF NOT EXISTS events (
202             id INTEGER PRIMARY KEY AUTOINCREMENT,
203             segment_id INTEGER NOT NULL,
204             timestamp REAL NOT NULL,
205             type TEXT NOT NULL,
206             player TEXT,
207             details TEXT,
208             FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
209         )
210     """)
211
212     def _migrate_to_v1(self, cursor):
213         # v1: raw candidate detections (pre-confirmation), detector-agnostic.
214         # Common fields are columns; detector-specific metrics go in `stats` JSON,
215         # so a new segmenter reuses this table by setting its own `detector`/`stats`.
216         cursor.execute("""
217             CREATE TABLE IF NOT EXISTS candidate_segments (
218                 id INTEGER PRIMARY KEY AUTOINCREMENT,
219                 video_id TEXT NOT NULL,
220                 detector TEXT NOT NULL,
221                 chunk_index INTEGER,
222                 start_time REAL NOT NULL,
223                 end_time REAL NOT NULL,
224                 duration REAL NOT NULL,
225                 confidence REAL,
226                 stats TEXT,
227                 detection_params TEXT,
228                 confirmed_segment_id INTEGER,
229                 human_label TEXT,
230                 notes TEXT,
231                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
232                 FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
233                 FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
DELETE SET NULL
234             )
235         """)
236         cursor.execute("""
237             CREATE INDEX IF NOT EXISTS idx_candidates_video_detector

```

```

238         ON candidate_segments(video_id, detector)
239     """)
240     cursor.execute("PRAGMA user_version = 1")
241
242     def _migrate_to_v2(self, cursor):
243         # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
244         # /api/process request. `status` is the EXECUTION state of the job
245         # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
246         # that failed validation) lives inside `result` JSON as result["status"].

```

19. assistant (2026-06-22T14:45:01.683Z)

Now let me read the jobs table schema and the key Database methods: add_play_segment, query_play_segments, create_job, claim_due_job, mark_job_done, get_job.

20. user (2026-06-22T14:45:03.196Z)

```

246         # that failed validation) lives inside `result` JSON as result["status"].
247         # `result` is the full sync-era response dict (incl. report paths), so the
248         # observability report is the job's artifact, per the plan.
249         cursor.execute("""
250             CREATE TABLE IF NOT EXISTS jobs (
251                 id TEXT PRIMARY KEY,
252                 video_path TEXT NOT NULL,
253                 video_id TEXT,
254                 segmenter TEXT,
255                 player_pool TEXT,
256                 max_frames INTEGER,
257                 status TEXT NOT NULL DEFAULT 'queued',
258                 progress TEXT,
259                 error TEXT,
260                 result TEXT,
261                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
262                 started_at TIMESTAMP,
263                 finished_at TIMESTAMP
264             )
265         """)
266         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
267         cursor.execute("PRAGMA user_version = 2")
268
269         def _migrate_to_v3(self, cursor):
270             # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the
271             # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
272             # due
273             # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker
274             # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
275             # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
276             # ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
277             # them.
278             self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
279             self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
TIMESTAMP")
280             cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
281             cursor.execute("PRAGMA user_version = 3")
282
283             def _migrate_to_v4(self, cursor):
284                 # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a NULLABLE
285                 # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
286                 # path can scope rows to an owner. **NULL = local install = byte-identical to
287                 # today** ? every existing row stays NULL and every existing query (which never
288                 # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows survive;
289                 # the ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
290                 # them.
291                 # Additive ONLY: no served-behavior change rides on this slice.

```

```

289         self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id
TEXT")
290         self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
user_id TEXT")
291         # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path free
292         # of index churn while making the eventual per-user lookups cheap.
293         cursor.execute(
294             "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
295             "WHERE user_id IS NOT NULL")
296         cursor.execute(
297             "CREATE INDEX IF NOT EXISTS idx_candidates_user "
298             "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
299         cursor.execute("PRAGMA user_version = 4")
300
301     def _migrate_to_v5(self, cursor):
302         # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per (user_id,
303         # version): the resolved per-user `fusion_config` overlay + an INLINE
304         # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
305         # and provenance that earned it. `is_active` pins the one row the serving bridge
306         # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`). This
slice
307         # only persists/loads it ? NOTHING reads it on the served path yet.
308         #
309         #   user_id           ? owner of the model (NULL allowed = the local install)
310         #   version           ? monotonic per-user model version
311         #   baseline_version  ? the shared baseline this was fit against (upgrade
switch)
312         #   feature_schema_hash ? fingerprint of the feature set; MINOR-vs-MAJOR re-fit
gate
313         #   fusion_config     ? JSON per-user FusionConfig overlay (cue_flags /
thresholds)
314         #   classifier_json   ? JSON LogisticRegression.to_dict() (None = config-only
model)
315         #   promotion_evidence ? JSON per_user_gate PromotionDecision (why it was
promoted)
316         #   n_videos_at_fit   ? held-out-user corpus size when fit (min_n trust floor
input)
317         #   is_active         ? the one resolved row for this user (partial-unique
below)
318         cursor.execute("""
319             CREATE TABLE IF NOT EXISTS user_models (
320                 id INTEGER PRIMARY KEY AUTOINCREMENT,
321                 user_id TEXT,
322                 version INTEGER NOT NULL DEFAULT 1,
323                 baseline_version TEXT,
324                 feature_schema_hash TEXT,
325                 fusion_config TEXT,
326                 classifier_json TEXT,
327                 promotion_evidence TEXT,
328                 n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
329                 is_active INTEGER NOT NULL DEFAULT 0,
330                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
331             )
332         """)
333         # One model version per user (NULLs compare distinct in SQLite, so the local
334         # install can still carry multiple versioned rows; the active-row guard below
335         # is what enforces a single served model).
336         cursor.execute("""
337             CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
338             ON user_models(user_id, version)
339         """)
340         # At most ONE active model per user: a partial-unique index over the active
rows.
341         # (SQLite treats NULL user_id rows as distinct, so the local install's single

```



```

342         # active row is still guarded ? there is one local user.)
343         cursor.execute("""
344             CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
345             ON user_models(user_id) WHERE is_active = 1
346         """)
347         cursor.execute("PRAGMA user_version = 5")
348
349     def _migrate_to_v6(self, cursor):
350         # v6: per-job COST LEDGER (COMPUTE_DECOUPLED_SERVING M8, D8). NULLABLE cost
columns on
351         # `jobs` (NULL = no cost recorded = byte-identical to today; nothing records
until
352         # billing.enabled + a real pricebook) + a VERSIONED `pricebook` table. Cost is
stored in
353         # USD (matches GCP billing = one source of truth); INR is a display conversion
at a
354         # configurable FX rate. ALTER (not recreate) so v1-v5 job rows survive; ADD
COLUMNs are
355         # idempotent so an interrupted upgrade can re-run them. Additive ONLY ? no
served change.

```

21. assistant (2026-06-22T14:45:06.232Z)

Now let me read the key Database methods: add_play_segment, query_play_segments, create_job, claim_due_job, mark_job_done, get_job.

22. user (2026-06-22T14:45:07.562Z)

```

405     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
406         """Register or update a video row WITHOUT touching its child segments.
407
408         Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
409         with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
410         That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
411         before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
412         row in place; clearing segments is now an explicit, deliberate operation
413         (clear_video_data / prune_segments).
414         """
415         return self._execute_write(
416             "INSERT INTO videos (id, filepath, status, validation_results) VALUES (?, ?,
?, ?) "
417             "ON CONFLICT(id) DO UPDATE SET "
418             "filepath=excluded.filepath, status=excluded.status, "
419             "validation_results=excluded.validation_results",
420             (video_id, filepath, status, json.dumps(validation_results)),
421             "Failed to add video",
422         )
423
424     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
425         """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).
426
427         Captured before a (re)segmentation so the run's new rows (id > watermark) can be
428         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
429         """
430         with self._connect() as conn:
431             cur = conn.cursor()
432             p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
433                             (video_id,)).fetchone()[0]
434             c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE

```

```

video_id = "?",
435             (video_id,)).fetchone()[0]
436         return int(p), int(c)
437
438     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
439                       play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
440                       cand_after: Optional[int] = None) -> None:
441         """Delete play/candidate segments by id range (events cascade from
play_segments).
442
443         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
444         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
rows
445         and keep the OLD ones when the run failed/produced nothing (`*_after`).
446         """
447         with self._connect() as conn:
448             cur = conn.cursor()
449             if play_upto is not None:
450                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
451             if play_after is not None:
452                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
453             if cand_upto is not None:
454                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
455             if cand_after is not None:
456                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
457             conn.commit()
458
459     def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
460         try:
461             with self._connect() as conn:
462                 cursor = conn.cursor()
463                 cursor.execute(
464                     "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
465                     (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
466                 )
467                 conn.commit()
468                 return cursor.lastrowid
469         except Exception as e:
470             logger.error(f"Failed to add play segment: {e}")
471             return None
472
473     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
474         return self._execute_write(
475             "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
476             (segment_id, timestamp, event_type, player, json.dumps(details or {})),
477             "Failed to add event",
478         )
479
480     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
481                               chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
482                               stats: Optional[Dict[str, Any]] = None,
483                               detection_params: Optional[Dict[str, Any]] = None) ->

```

```

Optional[int]:
484         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
485         detector-specific dicts stored as JSON. Returns the new candidate id, or
486         None."""
487         try:
488             with self._connect() as conn:
489                 cursor = conn.cursor()
490                 cursor.execute(
491                     """INSERT INTO candidate_segments
492                     (video_id, detector, chunk_index, start_time, end_time, duration,
493                     confidence, stats, detection_params)
494                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
495                     (video_id, detector, chunk_index, start_time, end_time, end_time -
496                     start_time,
497                     confidence, json.dumps(stats or {}), json.dumps(detection_params or
498                     {})))
499                 conn.commit()
500                 return cursor.lastrowid
501         except Exception as e:
502             logger.error(f"Failed to add candidate segment: {e}")
503             return None
504
505     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
506         """Record that a candidate detection was confirmed as a given play_segment."""
507         return self._execute_write(
508             "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id = ?",
509             (segment_id, candidate_id),
510             f"Failed to link candidate {candidate_id} to segment {segment_id}",
511         )
512
513     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
514                                 only_unlabeled: bool = False) -> List[Dict[str, Any]]:
515         """Fetch candidate detections for the annotation / fine-tuning workflow.
516         `stats` and `detection_params` are deserialized from JSON."""

```

23. assistant (2026-06-22T14:45:10.325Z)

Now the job lifecycle methods: query_play_segments, create_job, claim_due_job, mark_job_done, get_job.

24. user (2026-06-22T14:45:11.360Z)

```

564     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
565     List[Dict[str, Any]]:
566         """
567         Dynamically queries play segments based on filters:
568         - duration_min: float
569         - server: string
570         - receiver: string
571         - event_type: string (e.g. smash, foul)
572         """
573         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
574         params = [video_id]
575
576         if filters.get("duration_min"):
577             query += " AND r.duration >= ?"
578             params.append(filters["duration_min"])
579
580         if filters.get("server"):
581             query += " AND r.server = ?"
582             params.append(filters["server"])
583
584         if filters.get("receiver"):
585             query += " AND r.receiver = ?"

```

```

585         params.append(filters["receiver"])
586
587     if filters.get("event_type"):
588         # Check if there is an event of a specific type inside this segment
589         query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
590         params.append(filters["event_type"])
591
592     segments_list = []
593     with self._connect() as conn:
594         cursor = conn.cursor()
595         rows = cursor.execute(query, params).fetchall()
596
597         for row in rows:
598             r_dict = dict(row)
599             r_id = r_dict["id"]
600             r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
601
602             # Fetch associated events
603             event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
604             r_dict["events"] = []
605             for er in event_rows:
606                 e_dict = dict(er)
607                 e_dict["details"] = json.loads(e_dict["details"] or "{}")
608                 r_dict["events"].append(e_dict)
609
610             segments_list.append(r_dict)
611
612     return segments_list
613
614     def clear_video_data(self, video_id: str):
615         with self._connect() as conn:
616             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
617             conn.commit()
618
619     # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
620     # Writers follow the repo convention (swallow + logger.error, return bool); the
621     # worker logs loudly on a failed transition so a job can't silently wedge.
622
623     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
624                   player_pool: Optional[List[str]] = None,
625                   max_frames: Optional[int] = None,
626                   policy: Optional[Dict[str, Any]] = None,
627                   owner_id: Optional[str] = None,
628                   locale: Optional[str] = None) -> bool:
629         """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
630         (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
``owner_id``
631         (M9, D9) is the verified tenant from edge auth ? NULL for the local single-user
(default).
632         ``locale`` (v7) is the UI language the submission came from ? NULL when
unrecorded."""
633         return self._execute_write(
634             "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,
status, policy, "
635             "owner_id, locale) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?, ?)",
636             (job_id, video_path, segmenter,
637              json.dumps(player_pool) if player_pool is not None else None,
638              max_frames, json.dumps(policy) if policy is not None else None, owner_id,
639              locale),
640             f"Failed to create job {job_id}",
641

```

```

642     def count_jobs_by_locale(self) -> Dict[str, int]:
643         """PMF analytics: how many jobs were submitted per UI locale. Unrecorded locales
644         (NULL ? CLI / pre-v7 / older clients) bucket under '(unset)'. Returns {locale: count},
645         busiest first."""
646         with self._connect() as conn:
647             rows = conn.cursor().execute(
648                 "SELECT COALESCE(locale, '(unset)') AS loc, COUNT(*) AS n "
649                 "FROM jobs GROUP BY loc ORDER BY n DESC, loc"
650             ).fetchall()
651             return {str(r[0]): int(r[1]) for r in rows}
652
653     def mark_job_running(self, job_id: str) -> bool:
654         return self._execute_write(
655             "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
656             "progress='starting' WHERE id = ?", (job_id,),
657             f"Failed to mark job {job_id} running",
658         )
659
660     def set_job_progress(self, job_id: str, progress: str) -> bool:
661         return self._execute_write(
662             "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id),
663             f"Failed to set progress for job {job_id}",
664         )
665
666     def set_job_video_id(self, job_id: str, video_id: str) -> bool:
667         return self._execute_write(
668             "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id),
669             f"Failed to set video_id for job {job_id}",
670         )
671
672     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
673         """Job EXECUTED to completion. The pipeline's own verdict (success/failed
674         validation) is inside `result` ? job status 'done' means 'the worker
675         finished'."""
676         return self._execute_write(
677             "UPDATE jobs SET status='done', progress='finished', result=?, "
678             "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
679             (json.dumps(result), job_id),
680             f"Failed to mark job {job_id} done",
681         )
682
683     def mark_job_failed(self, job_id: str, error: str) -> bool:
684         """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
685         return self._execute_write(
686             "UPDATE jobs SET status='failed', error=?, "
687             "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id),
688             f"Failed to mark job {job_id} failed",
689         )
690
691     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
692         """Fetch one job; `result`/`player_pool` JSON deserialized."""
693         with self._connect() as conn:
694             row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
695             (job_id,)).fetchone()
696             if not row:

```

25. assistant (2026-06-22T14:45:15.642Z)

Now let me read `claim\_due\_job` to complete the status-transition picture, and confirm `mark\_job\_running` is called inside it.

26. user (2026-06-22T14:45:16.743Z)

```

831     def claim_due_job(self) -> Optional[Dict[str, Any]]:

```

```

832         """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
833         ``next_poll_at`` is due ?
834         flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
835         never
836         double-claim (the DB row-claim is the only coordination needed; no master).
837         Earliest-due
838         first. Returns the claimed job dict, or None if nothing is runnable."""
839         try:
840             with self._connect() as conn:
841                 cur = conn.cursor()
842                 row = cur.execute(
843                     "SELECT id, status FROM jobs WHERE status='queued' "
844                     "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
845                     CURRENT_TIMESTAMP)) "
846                     "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
847                     1").fetchone()
848                 if not row:
849                     return None
850                 cur.execute(
851                     "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
852                     "WHERE id=? AND status=?", (row["id"], row["status"]))
853                 conn.commit()
854                 if cur.rowcount != 1:
855                     return None # lost the race to a concurrent
856                 worker
857                 return self.get_job(row["id"])
858             except Exception as e:
859                 logger.error(f"Failed to claim a due job: {e}")
860                 return None
861
862     def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
863         """Crash recovery: any job still queued/running from a previous process is dead
864         (the executor is in-process). Mark failed so clients see a terminal state, not a
865         hang. Called once at startup. Returns the number of jobs swept."""
866         try:
867             with self._connect() as conn:
868                 cur = conn.cursor()
869                 cur.execute(
870                     "UPDATE jobs SET status='failed', error=?,
871                     finished_at=CURRENT_TIMESTAMP "
872                     "WHERE status IN ('queued', 'running')", (reason,))
873                 conn.commit()
874                 return cur.rowcount
875             except Exception as e:
876                 logger.error(f"Failed to sweep stale jobs: {e}")
877                 return 0
878
879     # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) -----
880     # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
881     # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
882     # serving.py) resolves them, and wiring that bridge into the production decision
883     # seam
884     # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
885     # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
886     # JSON-serialised dicts; the rest are scalar columns.
887
888     def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,
889                           baseline_version: Optional[str] = None,
890                           feature_schema_hash: Optional[str] = None,
891                           fusion_config: Optional[Dict[str, Any]] = None,
892                           classifier_json: Optional[Dict[str, Any]] = None,
893                           promotion_evidence: Optional[Dict[str, Any]] = None,
894                           n_videos_at_fit: int = 0,
895                           is_active: bool = False) -> Optional[int]:
896         """Insert or replace ONE per-user model row keyed by ``(user_id, version)``.

```

```

889
890         Idempotent on that key (UPSERT, not INSERT-OR-REPLACE: REPLACE would re-allocate
the
891         rowid). When ``is_active`` is True this row becomes the user's single active
model ?
892         any previously-active row for the SAME user is first cleared, so the
893         ``idx_user_models_one_active`` partial-unique invariant always holds (one active
model
894         per user). Returns the row id, or None on failure.
895         """
896         try:
897             with self._connect() as conn:
898                 cur = conn.cursor()
899                 if is_active:
900                     # Demote the current active row first (NULL user_id = local: IS NULL
match).
901                     if user_id is None:
902                         cur.execute(
903                             "UPDATE user_models SET is_active = 0 "
904                             "WHERE user_id IS NULL AND is_active = 1 AND version != ?",
905                             (version,))
906                     else:
907                         cur.execute(
908                             "UPDATE user_models SET is_active = 0 "
909                             "WHERE user_id = ? AND is_active = 1 AND version != ?",
910                             (user_id, version))
911                 cur.execute(
912                     """INSERT INTO user_models
913                     (user_id, version, baseline_version, feature_schema_hash,
914                     fusion_config, classifier_json, promotion_evidence,
915                     n_videos_at_fit, is_active)
916                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
917                     ON CONFLICT(user_id, version) DO UPDATE SET
918                     baseline_version=excluded.baseline_version,
919                     feature_schema_hash=excluded.feature_schema_hash,
920                     fusion_config=excluded.fusion_config,

```

27. assistant (2026-06-22T14:45:20.311Z)

I now have the complete picture. Let me check the `compute` directory to understand where a cloud-dispatch branch is intended to slot in, since the task mentions "this is where the cloud-dispatch branch must slot in."

28. user (2026-06-22T14:45:21.858Z)

```

__init__.py
base.py
cloud_run.py
---
__init__.py
__main__.py

```

29. user (2026-06-22T14:45:22.850Z)

```

Found 3 files
C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\cloud_run.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\__init__.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\base.py

```

30. assistant (2026-06-22T14:45:26.040Z)

There's already a `compute/` registry with `base.py` and `cloud\_run.py`. Let me check how it's wired (or not) relative to `\_execute\_processing`, since that determines exactly where the cloud-dispatch branch slots in.

31. user (2026-06-22T14:45:27.606Z)

```
=== base.py ===
53 backend/compute/base.py
155 backend/compute/cloud_run.py
61 backend/compute/__init__.py
269 total
```

32. user (2026-06-22T14:45:28.474Z)

```
44:from backend.utils.hashing import compute_video_id # canonical file-identity hashing
342:  """Honest probe of what THIS box can do ? the first-run wizard renders the compute step
from
369:          "compute_target": getattr(deployment, "compute_target", "") or "",
439:def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
459:    video_id = compute_video_id(local_video)
762:    # for a bucket URI (the same seam the process path uses, _execute_processing
main.py:159).
1013:    video_id = compute_video_id(local_video)
```

33. assistant (2026-06-22T14:45:33.053Z)

Confirmed: the `backend/compute/` registry (`base.py` + `cloud\_run.py`) exists but is **not** wired into `\_execute\_processing` or the job worker at all **?** `main.py` never imports `backend.compute`. The dispatch decision today is purely in-process. I have everything needed. Let me quickly confirm `base.py`'s shape so I can say precisely what exists to slot in.

34. user (2026-06-22T14:45:33.262Z)

```
1      """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3      COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4      (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5      core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6      (cloud-serving) ? without the core ever branching on the profile (D1).
7
8      The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9      non-local backend. Every other target (``"" / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
10     existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11     """
12
13     from __future__ import annotations
14
15     from abc import ABC, abstractmethod
16     from dataclasses import dataclass, field
17     from typing import Optional
18
19
20     @dataclass(frozen=True)
21     class ComputeJobSpec:
22         """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
23         from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
24         deterministic regardless of where the job ran."""
25
26         video_uri: str # input (gs://? for cloud; a path/local://
for local)
27         out_uri: str # output ROOT (gs://? bucket for cloud)
28         segmenter: Optional[str] = None
29         # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
```



```

frozen/hashable.
30         config_overrides: tuple[str, ...] = ()
31
32
33     @dataclass(frozen=True)
34     class ComputeResult:
35         """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
36         (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
37         empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
38         done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
39
40         returncode: int
41         outputs_uri: str = ""
42         video_id: str = ""
43         report: dict = field(default_factory=dict)
44
45
46     class ComputeBackend(ABC):
47         """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
48         its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
49         execution identically."""
50
51         @abstractmethod
52         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
53             """Run ``spec`` to completion and return its ``ComputeResult``."""
54

```

35. assistant (2026-06-22T14:46:26.963Z)

I have the complete map. Here is the exact in-process flow.

How the API job flow runs a job IN-PROCESS today

All file:line refs are in `bhi-cloudrun` (HEAD `df3b225`).

0. Submit ? `POST /api/process` creates a `queued` row, kicks the worker

```

`backend/main.py:559-608` (`process_video`):
- Fast-fail validation (path, file existence, unknown segmenter) ? immediate 4xx.
- `edge_auth.resolve_tenant(...)` ? `owner_id` (default-OFF ? `None`), `main.py:572`.
- Generates `job_id = uuid.uuid4().hex` (`main.py:599`), inserts a row via
`db_instance.create_job(...)` (`main.py:600-603`). Returns **202** `{"job_id",
"status":"queued", "poll":"/api/jobs/{job_id}"}` (`main.py:605-608`).
- Kicks the drain: `_get_executor().submit(_drain_due_jobs)` (`main.py:604`). The executor is
`ThreadPoolExecutor(max_workers=1)` (`backend/api/job_worker.py:33-38`) ? single-worker by
design.

```

```

`create_job` (`backend/infrastructure/database.py:623-640`) inserts into the `jobs` table with
`status='queued'`; schema is `_migrate_to_v2` at `database.py:249-267` (cols: `id`, `video_path`,
`video_id`, `segmenter`, `player_pool`, `max_frames`, `status`, `progress`, `error`, `result`, `created_at`,
`started_at`, `finished_at`), plus v3 `policy`/`next_poll_at` (`database.py:276-277`) and v6 cost
cols. `owner_id`/`locale` are later-migration cols.

```

1. Drain loop ? claim then run

```

`_drain_due_jobs` (`job_worker.py:132-153`): loops `db_instance.claim_due_job()` ?
`_run_claimed_job(job)` until `claim_due_job` returns `None`, then
`_schedule_next_drain_if_waiting()`.

```

`claim\_due\_job` (`database.py:831-854`) is the **queued ? running** transition: atomic compare-and-set ? SELECTs the earliest runnable job (`status='queued'` OR a `waiting` job whose `next\_poll\_at` is due, `database.py:840-842`), then `UPDATE jobs SET status='running', started\_at=CURRENT\_TIMESTAMP WHERE id=? AND status=?` (`database.py:845-847`). If `cur.rowcount != 1` it lost the race ? returns `None` (`database.py:849-850`). On success returns the full job dict via `get\_job` (`database.py:851`). (`mark\_job\_running` at `database.py:652-657` exists but the actual claim flips status inline here.)

2. Run one claimed job ? `\_run\_claimed\_job`

`job\_worker.py:72-98`. Resolves everything through `backend.main`:
- `req = \_main.\_job\_to\_request(job)` ? rebuilds a `ProcessRequest` from the row (`job\_worker.py:59-69`); `ProcessRequest` shape at `main.py:259-266` (`video\_path, player\_pool, max\_frames, segmenter, locale`).
- `result = \_main.\_execute\_processing(req, progress=lambda stage: db\_instance.set\_job\_progress(job\_id, stage))` (`job\_worker.py:86-87`). The progress callback writes the coarse stage string to `jobs.progress` (`set\_job\_progress`, `database.py:659-663`).
- On success: if `result["video\_id"]`, `db\_instance.set\_job\_video\_id(job\_id, ...)` (`job\_worker.py:88-89`, `database.py:665-669`); then `db\_instance.mark\_job\_done(job\_id, result)` (`job\_worker.py:90`) ? the **running ? done** transition; then `\_maybe\_record\_job\_cost` (M8, default-OFF no-op, `job\_worker.py:91`, `:101-129`).
- On `JobWaiting` (`job\_worker.py:92-95`): `set\_job\_waiting(...)` parks it (**running ? waiting**) ? not used by the production path today (`job\_worker.py:41-56`).
- On any other `Exception` (`job\_worker.py:96-98`): `db\_instance.mark\_job\_failed(job\_id, "{type}: {e}")` ? the **running ? failed** transition.

3. The pipeline ? `\_execute\_processing`

`backend/main.py:439-556`. This is the former synchronous `/api/process` body. Sequence:
1. `notify("hashing")` ? `storage.acquire\_local\_input(req.video\_path, ...)` resolves URI/path to a local file (`main.py:458`); `video\_id = compute\_video\_id(local\_video)` (MD5, `main.py:459`); `was\_reingest = db\_instance.get\_video(video\_id) is not None` (`main.py:460`).
2. `notify("validating")` ? `registry.run\_all\_with\_context(...)` (`main.py:465`). If any validator fails (`main.py:475-485`): `db\_instance.add\_video(video\_id, req.video\_path, "failed", [...])` (UPSERT, no cascade ? `database.py:405-422`), returns `{"video\_id", "status": "failed", "message", "results"}`.
3. Else: `db\_instance.add\_video(video\_id, req.video\_path, "processed", [...])` (`main.py:489`). Resolve segmenter: `seg\_name = req.segmenter or default\_seg` (`main.py:469-470`); `segmenter\_cls = segmenter\_registry.get(seg\_name)` (`main.py:491`).
4. **Destroy-after-confirm**: capture `play\_wm, cand\_wm = db\_instance.segment\_watermarks(video\_id)` (`main.py:510`, `database.py:424-436`). Build segmenter: `segmenter = segmenter\_cls(db\_instance, global\_config)` (`main.py:511`).
5. **Run the core**: `result = segmenter.process\_video(video\_id, local\_video, req.player\_pool, req.max\_frames)` (`main.py:513`). On raise ? `prune\_segments(... play\_after=play\_wm, cand\_after=cand\_wm)` drops new partial rows and re-raises (`main.py:514-519`). On `Failure` ? same prune + `status:"failed"` response (`main.py:522-532`).
6. `segments = result.value` (`main.py:534`); `notify("finalizing")`; `\_finalize\_reingest(video\_id, segments, play\_wm, cand\_wm)` (`main.py:536`, `:283-293`) ? on a non-empty run drops the PRIOR rows (`prune\_segments play\_upto=play\_wm`); on empty drops the NEW rows (keeps prior good data).
7. Returns the success dict (`main.py:538-544`): `{"video\_id", "status": "success", "message", "results": [validators], "play\_segments": segments}`.
8. `finally`: always emits the per-video observability report and grafts `response["reports"] = paths` (`main.py:546-556`).

How segments actually land in SQLite: `\_execute\_processing` does **not** insert segments itself. The segmenter (`segmenter.process\_video`, ABC at `backend/pipeline/segmenters/base.py:23-33`) writes them **as a side effect** via its injected `self.db`, calling `db\_instance.add\_play\_segment(video\_id, start, end, server, receiver, metadata)` (`database.py:459-471` ? `INSERT INTO play\_segments (...) VALUES (...)`, returns `lastrowid`) and optionally `add\_candidate\_segment` (`database.py:480-501`) / `add\_event` (`database.py:473-478`). The returned `result.value` list is the same segments, used for the count + watermark finalize. So persistence is: **segmenter ? `self.db.add\_play\_segment` ? `play\_segments` table**, with `\_execute\_processing` only managing the watermark/prune lifecycle around it.

result dict shape stored on the job

``mark_job_done(job_id, result)`` (``database.py:671-679``) does ``UPDATE jobs SET status='done', progress='finished', result=<json.dumps(result)>, finished_at=CURRENT_TIMESTAMP``. The stored ``result`` is exactly the ``_execute_processing`` return dict:

```
...
{"video_id", "status": "success|"failed", "message", "results": [...validators...],
 "play_segments": [...] (success/segmenter-fail branches), "reports": {...} (grafted in
finally))
...
```

Note ``result["status"]`` is the `**pipeline verdict**` (``success`/`failed``), distinct from the job's `**execution**` ``status`` column (``done``).

status transitions (jobs.status)

``queued`` (``create_job, `database.py:635``) ? ``running`` (``claim_due_job CAS, `database.py:846``) ? terminal ``done`` (``mark_job_done, `database.py:675``) `**or**` ``failed`` (``mark_job_failed`` on worker crash, ``database.py:684``) `**or**` parked ``waiting`` (``set_job_waiting, `database.py:802``, unused in prod) which re-enters ``claim_due_job`` when ``next_poll_at`` is due. Crash recovery: ``fail_stale_jobs`` (``database.py:856-870``) sweeps any ``queued`/`running`` rows to ``failed`` at startup (the in-process executor died with the process).

GET /api/query reads segments afterward

``POST /api/query`` (``main.py:694-703``, body ``QueryRequest`` at ``main.py:268-273``) calls ``db_instance.query_play_segments(req.video_id, filters)`` and returns ``{"play_segments": segments}``. ``query_play_segments`` (``database.py:564-612``) does ``SELECT r.* FROM play_segments r WHERE r.video_id = ?`` with optional ``duration_min`` / ``server`` / ``receiver`` / ``event_type`` (EXISTS-subquery on ``events``) filters, deserializes ``metadata`` JSON, and attaches each segment's ``events`` (``database.py:597-610``). It reads `**only**` ``play_segments`/`events`` keyed by ``video_id`` ? it has no knowledge of jobs. ``/api/compile`` (``main.py:705-717``) reads the same way and filters by the client-supplied ``segment_ids``.

Where the cloud-dispatch branch must slot in

The cloud-dispatch interface `**already exists but is NOT wired**`: ``backend/compute/`` has ``ComputeBackend`` ABC + ``ComputeJobSpec`/`ComputeResult`` (``backend/compute/base.py``, full file above) and ``cloud_run.py`` (155 lines). `**`backend/main.py` never imports `backend.compute`**` (grep confirmed ? only ``compute_video_id`` and ``compute_target`` from ``/api/capabilities`` appear). So today the dispatch decision is implicit: ``_execute_processing`` always runs the segmenter in-process on the worker thread.

The natural insertion point is `**`_execute_processing` at `main.py:511-513`**` ? right where it builds ``segmenter = segmenter_cls(db_instance, global_config)`` and calls ``segmenter.process_video(...)``. Per ``base.py:8-10``, the seam is gated on ``deployment.compute_target == "cloud_run"``: a factory returns a ``ComputeBackend`` (cloud) or ``None`` (run inline / byte-identical today). The cloud branch would replace the local ``process_video`` call with ``backend.run_infer(ComputeJobSpec(video_uri=..., out_uri=..., segmenter=seg_name, ...))``, then ingest the remote ``ComputeResult.outputs_uri`` artifacts back into ``play_segments`` so that the downstream contract ? the same ``result`` dict stored by ``mark_job_done``, and ``query_play_segments`` reading ``play_segments`` by ``video_id`` ? stays identical. The watermark/``_finalize_reingest`` lifecycle (``main.py:510, 536``) and the ``result["video_id"]`/`play_segments`` shape must be preserved for ``/api/query`` and ``/api/compile`` to keep working unchanged.

Relevant files (absolute):

- ``C:\Users\avidu\Projects\bhi-cloudrun\backend\api\job_worker.py`` ? worker drain/claim/run loop.
- ``C:\Users\avidu\Projects\bhi-cloudrun\backend\main.py`` ? ``_execute_processing`` (439-556), ``/api/process`` (559-608), ``/api/query`` (694-703), ``/api/compile`` (705-717), ``ProcessRequest`/`QueryRequest`` (259-273).
- ``C:\Users\avidu\Projects\bhi-cloudrun\backend\infrastructure\database.py`` ? jobs schema (249-279), ``add_video`/`segment_watermarks`/`prune_segments`/`add_play_segment`` (405-471), ``query_play_segments`` (564-612), job store (623-870).
- ``C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\segmenters\base.py`` ? ``process_video``

ABC contract (23-33).

- `C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\base.py` + `cloud\_run.py` ? the unwired
`ComputeBackend` dispatch seam the cloud branch plugs into.